

CHECKPOINT 3 - PROJECT REPORT

C- Compiler: Code generation and TM execution

Athina Manolakou, Ayesha Khan and Shifa Sheikh

1. WHAT HAS BEEN DONE

In this checkpoint we completed the C- compiler by implementing code generation, translating valid C- programs into TM assembly that can be executed on the TM simulator. Building on the scanner, parser, abstract syntax tree (AST), and semantic analysis from earlier checkpoints, the compiler now supports the full pipeline from source code to assembly output.

We implemented code generation for expressions and assignments, control structures (`if/else`, `while`), function calls (including self-recursion and mutual recursion where supported by the program structure), nested blocks, and arrays (including array parameters where applicable). We added runtime support consistent with the TM model (frame pointer, globals, locals, call stack) and runtime checks for array index out of bounds. A new option `-c` writes assembly to a `.tm` file next to the input (e.g. `file.cm` $\rightarrow$ `file.tm`).

The compiler can now be invoked as:

`./CM file.cm` - parsing + semantic analysis

`./CM -a file.cm` - outputs AST

`./CM -s file.cm` - outputs symbol table

`./CM -c file.cm` - generates TM assembly code

Output files are only produced when the program is free of syntax and semantic errors.

We tested the implementation on the provided sample programs (`fac.cm`, `booltest.cm`, `gcd.cm`, `sort.cm`, `mutual.cm`) as well as ten custom test files (`1.cm–0.cm`) that cover valid programs, syntax errors, semantic/type errors, and runtime errors such as array index out-of-bounds.

Programs 1–3 are valid and produce runnable assembly; 4–8 illustrate syntax, semantic, and runtime issues (at most three distinct error categories per file, per spec); 9 and 0 are unrestricted stress cases. Each test file includes a comment header describing what it exercises and, where relevant, what errors are expected.

We verified builds and runs on the course Linux environment (linux.socs.uoguelph.ca), which matches the grading and demo environment. The project includes a Makefile (make builds the CM driver; `make clean` removes generated artifacts) and a README with build and test instructions. The submission follows CourseLink expectations: source, Makefile, test programs, README, report, packaged as requested. A short demo with a TA is scheduled per the checkpoint instructions.

2. TECHNIQUES AND DESIGN

2.1 Code Generation

Code generation is implemented by traversing the AST with a dedicated visitor that emits TM instructions for each node, preserving the structure and semantics of the source program. The generated code uses a stack-oriented model: registers and memory hold intermediate values, activation records hold parameters, locals, temporaries, and linkage information, and a consistent calling convention is used for calls and returns.

Jump scheduling: Conditional and loop constructs use forward branches implemented by reserving instruction slots and patching jump targets once the extent of a branch or loop is known (sometimes described as backpatching or forward patching in compiler texts).

2.2 Runtime Environment

The runtime layout follows the TM simulator conventions discussed in the course notes: a global pointer for globals, a frame pointer for each activation, storage for return address and saved

frame pointer, and a stack of frames for nested calls. Each function call sets up a new activation record; return restores the caller's frame and resumes at the saved return point.

2.3 Expressions and Assignments

Arithmetic and relational expressions lower to TM register and ALU operations. Binary expressions are handled by generating code for operands then applying the operator.

Assignments store the evaluated value into the correct global or frame offset; array element assignment computes $\text{base} + \text{index}$ (with bounds checks where implemented) before storing.

2.4 Control Structures

If-then-else uses conditional branches to skip or execute branches based on the condition (integer/boolean zero vs non-zero as appropriate). While loops use a top-tested loop with a backward branch to the condition.

2.5 Functions and Recursion

Calls evaluate arguments, establish the callee frame, transfer control to the function entry, and leave the return value in the agreed location (typically the accumulator). Self-recursion (e.g. GCD-style examples) and mutual recursion rely on every function having a known entry address and consistent frame layout; function prototypes are integrated with the symbol table so a definition can complete a forward declaration without duplicate-definition errors.

2.6 Arrays and Nested Blocks

Arrays are laid out in memory with a known base; indexing computes the address of the element. Nested blocks allocate inner locals in the current frame using scope-aware offsets so inner declarations shadow outer names correctly within their region.

2.7 Runtime Error Handling

For indexed accesses, the compiler emits checks so that out-of-range indices are detected at run time on the TM machine; on failure the program halts in a controlled way rather than silently corrupting memory. Other runtime situations (e.g. division by zero) are not fully guarded unless noted below.

2.8 Integration with Earlier Phases

Code generation runs only after successful parsing and semantic analysis. Earlier checkpoints remain essential:

- **Scanning and parsing:** The lexer recognizes the major token classes for C-; the parser builds an AST. The parser includes error recovery so that a single mistake does not always abort the entire parse (recovery is basic to moderate in scope—enough for demonstration and incremental testing, not an exhaustive error-correction engine).
- **Semantic / type checking:** A symbol table (hash-based scope structure) records declarations and supports checks for undefined and redefined identifiers, operand types for operators, assignment compatibility, if/while test types, array indices, function calls and return types, and prototype vs definition consistency. Where possible, the analyzer continues after some errors to report multiple issues (type-check recovery), though severely ill-typed programs are not codegen'd.

This division matches the Checkpoint Three marking scheme: codegen is the capstone, but scanning, parsing, and type checking remain graded components of the full compiler.

2.9 Evolution from Checkpoint 1 and Checkpoint 2

Across the project, the design moved from lexing and parsing with AST output (CP1), to symbol tables and static semantics (CP2), to TM code generation and runtime checks (CP3). The AST

and visitor pattern introduced early became the stable interface for semantic analysis and then for instruction selection. Scoping rules designed for CP2 were reused for frame layout and offset assignment in CP3. The main structural change in CP3 was adding the code generator visitor and `-c` output path in `Main.java`, plus tightening semantic rules (e.g. arrays, calls, prototypes) so that generated code is sound.

3. LESSONS LEARNED

- **Mapping high-level constructs to low-level instructions:** Translating structured constructs such as loops and function calls into jump-based assembly required careful planning and testing.
- **Managing activation records:** Implementing function calls and recursion highlighted the importance of consistent stack frame design.
- **Debugging generated code:** Errors in code generation were difficult to trace; we had to use small test cases and inspect generated TM code.
- **Integration complexity:** Ensuring that all compiler phases (scanner, parser, semantic analysis, and code generation) work together seamlessly was complicated.

4. ASSUMPTIONS AND LIMITATIONS

We treat the course C- grammar as authoritative. Input is expected to use the `.cm` extension. Code generation is only performed when there are no syntax or semantic errors sufficient to block a safe translation.

The TM model has fixed memory; we do not perform optimization (inlining, register allocation across calls, etc.). Local array declarations within functions and nested compound blocks are not currently supported in the code generation phase. Arrays are fully supported as global variables and as function parameters, which aligns with the required functionality demonstrated in the

provided test programs ([sort.cm](#)). Runtime checking is focused on array bounds as required, division by zero and similar issues are not all explicitly trapped. Error recovery in the parser and type checker improves diagnostics but does not guarantee compilation of arbitrary broken programs.

5. POSSIBLE IMPROVEMENTS

- Add support for line comments in the scanner.
- Integrate boolean operators into the expression grammar.
- Enhance error messages to suggest expected tokens (e.g., “missing semicolon”).
- Add more recovery points for contexts like parenthesised expressions/malformed blocks.
- Provide a debug mode to display the token stream during scanning.
- Extend code generation to support local array declarations within functions and nested compound blocks.

6. CONTRIBUTIONS

Ayesha Khan implemented the code generation foundation: `-c` wiring in [Main.java](#), prelude and finale, I/O hooks, emitter utilities, and arithmetic and assignment codegen, establishing conventions for globals, locals, and instruction emission used by the rest of the generator.

Athina Manolakou implemented control flow and function-related codegen: `if/else`, `while`, function calls, returns, recursion, and nested block / scope layout on the frame.

Shifa Sheikh implemented arrays and array parameter codegen, runtime out-of-bounds handling, integration across phases, the [1.cm–0.cm](#) test suite and validation, and submission materials (README alignment, report assembly, and packaging support as required by the team).